

PROJECT DEEPWATER

The Product Capability Realization and Systems Audit Program

Governing Architecture, Audit Methodology, Realization Ledger,
Cross-Project Remediation, and Continuous Product-Truth Requirements

Governing Principle
Every meaningful Voyagewright capability must either be intentionally internal, security-restricted, operational, or API-only by design, or it must trace all the way from canonical backend truth to a complete human-facing realization. Deepwater identifies, classifies, assigns, and proves gaps; it does not steal canonical ownership from the subsystem it audits.

Version 1.0 | August 8, 2026

Prepared for the Voyagewright platform

Document Control

Field	Value
Program	Project Deepwater
Formal subtitle	The Product Capability Realization and Systems Audit Program
Document type	Governing architecture, audit methodology, and product-realization requirements
Version	1.0
Date	August 8, 2026
Status	Governing baseline
Preparation baseline	Latest reviewed integrated mainline at document preparation: f1c2f22dd935322c1a71eb80c51592f243dc196d; execution must revalidate current origin/main
Primary sources	Feature Catalog; Global Product Governance; Project Homeport artifacts; Sounding Line; Continuous Development and Mainline Integration Standard; canonical subsystem governing documents
Core implementation rule	Deepwater audits and assigns capability gaps. Canonical subsystem owners implement domain behavior. Deepwater may own cross-layer integration contracts and product-realization evidence.

Authority and Precedence

Authority	What it controls
Current fetched origin/main	Implementation truth: actual code, routes, services, schema, tests, and accepted behavior
Voyagewright Global Product Governance	Human-facing product invariants, route reachability, screen maturity, natural journeys, owner acceptance
This Deepwater document	Capability inventory, realization methodology, ledger semantics, gap classification, assignment, and closure proof
Project Sounding Line	Verification planning, evidence validity, test execution, mainline and release decisions
Continuous Development Standard	Phase boundaries, concurrency, dependencies, prompt convergence, mainline safety
Subsystem governing documents	Canonical ownership and product behavior inside Wayfarer, One Voyage, Harborlight, Drydock, Helm, Wakebook, Admiralty, Shipwright, Tideglass, Sealed Hold, Landfall, etc.

Contents

1. Executive Summary
2. Project Identity and Product Vision
3. Why Deepwater Exists
4. Non-Negotiable Principles
5. Scope and Explicit Non-Goals
6. Canonical Ownership Boundary
7. Relationship to Feature Catalog and Homeport
8. Capability Realization Model
9. Capability Realization Ladder
10. Classification Taxonomy
11. Gap and Loss Taxonomy
12. Canonical Capability Trace
13. Backend and Domain Audit Method
14. Service, API, and Action Audit Method
15. Projection and DTO Audit Method
16. Frontend and Interaction Audit Method
17. Discoverability and Navigation Audit Method
18. Screen-State, Feedback, and Recovery Audit
19. Accessibility, Responsive, and Motion Audit
20. Security, Privacy, and Restricted-Capability Audit
21. Documentation and Feature-Catalog Truth Audit
22. Capability Realization Ledger
23. Finding, Assignment, and Ownership Workflow
24. Remediation Strategy and Deepwater Slices
25. Cross-Project Coordination and Concurrency
26. Sounding Line Verification and Evidence
27. Machine-Readable Control Plane
28. Metrics, Scoring, and Prioritization
29. Five-Phase Implementation Program
30. Final Program Acceptance and Continuous Governance
- Appendix A. Capability Realization Ledger Schema
- Appendix B. Capability Trace Worksheet

Appendix C. Finding and Assignment Record

Appendix D. Phase 1 Audit Checklist

Appendix E. Example Capability Traces

Appendix F. Mainline Safety Contracts by Phase

Appendix G. Final Acceptance Checklist

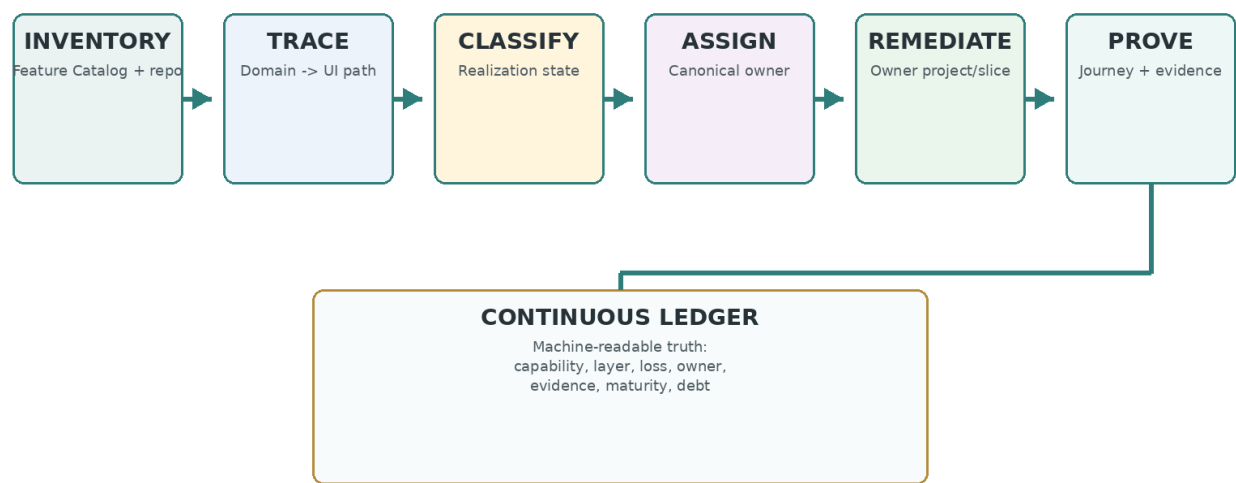
1. Executive Summary

Project Deepwater is the permanent Voyagewright program for determining whether the product actually exposes and uses the capabilities its architecture already possesses. It exists because a backend capability, database model, service, API, or passing test does not automatically become a useful product feature. The platform has already demonstrated the inverse of the more common software failure: sophisticated backend systems can exist while the human-facing surface exposes only a fraction of their value.

Deepwater creates one authoritative method for tracing each meaningful capability from canonical domain truth through service execution, authorized transport, projection, user interface, discoverability, complete interaction states, accessibility, whole-journey proof, and final product acceptance. It records where capability is lost, hidden, duplicated, degraded, or intentionally restricted.

The program is not a one-time audit report and is not permission to redesign every subsystem. Its permanent output is a machine-readable Capability Realization Ledger that can be regenerated and reconciled as Voyagewright evolves. Each gap is assigned to the canonical project that owns the affected behavior. Deepwater owns the audit methodology, cross-layer realization contract, ledger, prioritization, and closure proof.

Project Deepwater: Capability Realization Pipeline



Deepwater never becomes the implementation owner of every subsystem; it identifies, assigns, and proves reali

Figure 1. Deepwater follows capability from inventory through proof while preserving canonical subsystem ownership.

Core Product Outcome
For every meaningful capability, Voyagewright can answer: What exists? Where is it exposed? What information is lost? Can a human discover and use it? Are all required states complete? Who owns any gap? What evidence proves the gap is closed?

2. Project Identity and Product Vision

2.1 Program name

The subsystem is named Project Deepwater. The name reflects work performed below the visible surface: tracing architecture, services, projections, APIs, and implementation seams that ordinary users cannot see but whose quality determines whether the visible product is genuinely capable.

2.2 Formal subtitle

The Product Capability Realization and Systems Audit Program.

2.3 Product vision

Deepwater should make the relationship between backend capability and human experience explicit enough that neither Codex nor a human developer can honestly claim a feature is complete while most of its useful behavior remains stranded in services, DTOs, undocumented routes, or hidden controls.

The product vision is not 'put a button on every backend method.' Some capabilities are intentionally internal, dangerous, support-only, administrative, security-sensitive, or suitable only for machine consumers. Deepwater therefore evaluates appropriateness, not raw exposure. The requirement is intentionality: every meaningful capability receives an explicit realization disposition.

2.4 Primary use cases

- Audit an entire subsystem after a major project closes and determine which accepted backend capabilities remain under-realized.
- Trace a Feature Catalog entry to its real screen, action, state model, navigation path, and evidence.
- Detect useful API fields that are discarded by server projections or client DTOs.
- Find backend mutations that are exposed only as read-only UI, or UI controls whose backend operation is missing.
- Identify dead routes, dead controls, hidden features, duplicate authorities, and documentation claims that exceed product reality.
- Assign each gap to its canonical owner instead of allowing a cross-cutting audit project to become a second implementation authority.
- Prevent regression by requiring new capabilities to declare a realization disposition and evidence before they are represented as product-complete.

3. Why Deepwater Exists

Project Homeport proved that Voyagewright can contain substantial valid backend and domain work while the running product still fails basic human expectations. Homeport repaired the most severe identity, shell, reachability, Personal Harbor, Community Harbor, screen-state, and journey problems. Deepwater exists because those recovery findings revealed a broader systemic risk: capability loss between layers.

A capability can be lost in many places. A service may support an operation that no route calls. An API may return rich metadata that a DTO strips away. A DTO may expose information that the interface never renders. The interface may render a control that users cannot discover. A route may be discoverable but lack failure recovery. A feature may work only through a developer path or direct URL. A polished screen may represent only one of five backend states.

Deepwater treats these as first-class engineering nonconformities rather than subjective polish concerns. The program makes the loss measurable and assigns it to an owner while the context is still available.

Failure pattern	Example Deepwater question
Backend-only capability	Can the server perform an operation that no authorized human surface exposes?
Projection loss	Does the backend know provenance, state, or diagnostics that the client never receives?
Interaction loss	Does the API support a mutation while the UI exposes only viewing?
Discoverability loss	Can the feature be reached only through a direct URL or hidden link?
State loss	Are pending, conflict, recovery, offline, revoked, or empty states collapsed into generic failure?
Documentation mismatch	Does the Feature Catalog call something complete when the product surface is skeletal or partial?
Duplicate authority	Do two layers or legacy adapters independently decide the same business truth?

4. Non-Negotiable Principles

Principle	Requirement
Intentional Realization	Every meaningful capability must be explicitly realized or explicitly classified internal/restricted by design.
Canonical Ownership	Deepwater does not create a competing implementation authority. Gaps are assigned to the subsystem that owns the behavior.
Product Reality Over Layer Count	More backend layers do not increase maturity if humans still cannot use the capability.
Traceability End to End	Every audited capability must trace through domain, service, transport, projection, UI, discoverability, states, and evidence as applicable.
No Cosmetic Completion	A button or route does not close a capability gap if states, permissions, recovery, feedback, or journey integration remain incomplete.
No Forced Exposure	Secrets, internal operations, destructive controls, and support-only details remain protected when product exposure would be unsafe or confusing.
Natural Discovery	Ordinary capabilities must be reachable through visible, logical product paths, not remembered URLs.
Evidence Bound to Source	Realization claims must identify the exact source state, route/screen, fixtures, and verification evidence.
Current Main Is Truth	Audits target freshly fetched accepted main, not historic completion receipts or remembered architecture.
Continuous Ledger	Deepwater leaves a durable machine-readable ledger that future work updates instead of restarting the audit from folklore.

5. Scope and Explicit Non-Goals

5.1 Deepwater owns

- Capability inventory and stable capability identifiers.
- End-to-end capability traces and layer coverage.
- Capability-realization classification and maturity.
- Gap taxonomy, severity, confidence, and impact.
- Canonical owner assignment and remediation routing.
- Cross-layer integration defects that do not belong cleanly to one domain owner.
- Machine-readable Capability Realization Ledger and audit tooling.
- Feature Catalog realization metadata and product-maturity reconciliation.
- Deepwater-specific dashboards, reports, drift checks, and regression gates.
- Program-level proof that assigned realization gaps are closed.

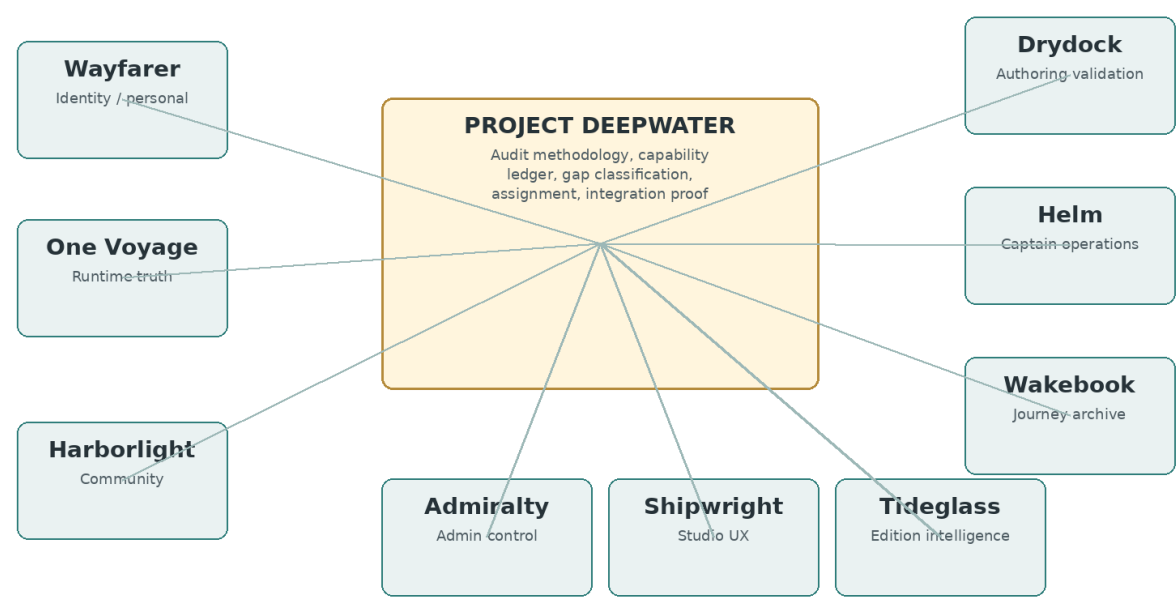
5.2 Deepwater does not own

- Account identity or privacy truth owned by Wayfarer.
- Chronicle authoring semantics owned by Creator Studio/Drydock/One Voyage.
- Live Voyage progression owned by One Voyage.
- Community business rules owned by Harborlight.
- Captain runtime authority owned by Helm.
- Personal journey-history semantics owned by Wakebook.
- Administrative authority owned by Admiralty.
- Edition-difference semantics owned by Tideglass.
- Private-content security owned by Sealed Hold.
- Animation/runtime presentation truth owned by Lanternwake.
- Geospatial truth owned by Landfall.
- Release verification authority owned by Sounding Line.

Hard Boundary
Deepwater may implement cross-layer adapters, missing projections, route integration, or realization tooling when explicitly assigned. It must not silently fork canonical domain logic to make an audit row turn green.

6. Canonical Ownership Boundary

Deepwater Ownership Boundary



Rule: Deepwater may repair cross-layer integration gaps, but canonical domain behavior remains with the owning proje

Figure 2. Deepwater sits across subsystem boundaries without becoming their business-logic owner.

Every finding must name a canonical owner before remediation begins. When a gap crosses layers, Deepwater may name a primary owner and one or more contributing owners. A route owned by Shipwright may expose a projection owned by Drydock and data owned by One Voyage; the finding must preserve all three responsibilities rather than selecting whichever team owns the easiest file to edit.

Ambiguous ownership is itself a Deepwater finding. Closure is blocked until the domain authority is resolved. Git file location is evidence, not authority: a React component living beside an API route does not make the component the owner of backend semantics.

7. Relationship to Feature Catalog and Homeport

The Feature Catalog is Deepwater's most important inventory source because it records accepted meaningful platform capabilities. Deepwater must not assume the catalog is exhaustive or perfectly truthful; it must reconcile catalog entries against implementation and may discover uncataloged capabilities or overclaimed maturity.

Homeport established route inventories, screen contracts, natural journeys, nonconformities, product states, and owner walkthroughs. Deepwater consumes those artifacts rather than rebuilding them. Homeport asked whether the product was coherent; Deepwater asks whether every meaningful capability reaches the coherent product at the depth it deserves.

Existing authority	Deepwater use
Feature Catalog	Seed capability inventory, feature IDs, ownership, mainline status, user entry points
Homeport route inventory	Discoverability and route-path proof
Homeport screen catalog	Screen maturity, required states, viewport evidence
Homeport journey catalog	Natural end-to-end workflow evidence
Global Product Governance	Definition of product reality and owner acceptance
Sounding Line	Source-bound proof and acceptance decisions

8. Capability Realization Model

Deepwater models a capability as a stable product or platform ability with a canonical owner, one or more consumers, a realization disposition, and an auditable chain from source truth to experience. A capability is more specific than a project and broader than a single function.

8.1 Capability identity

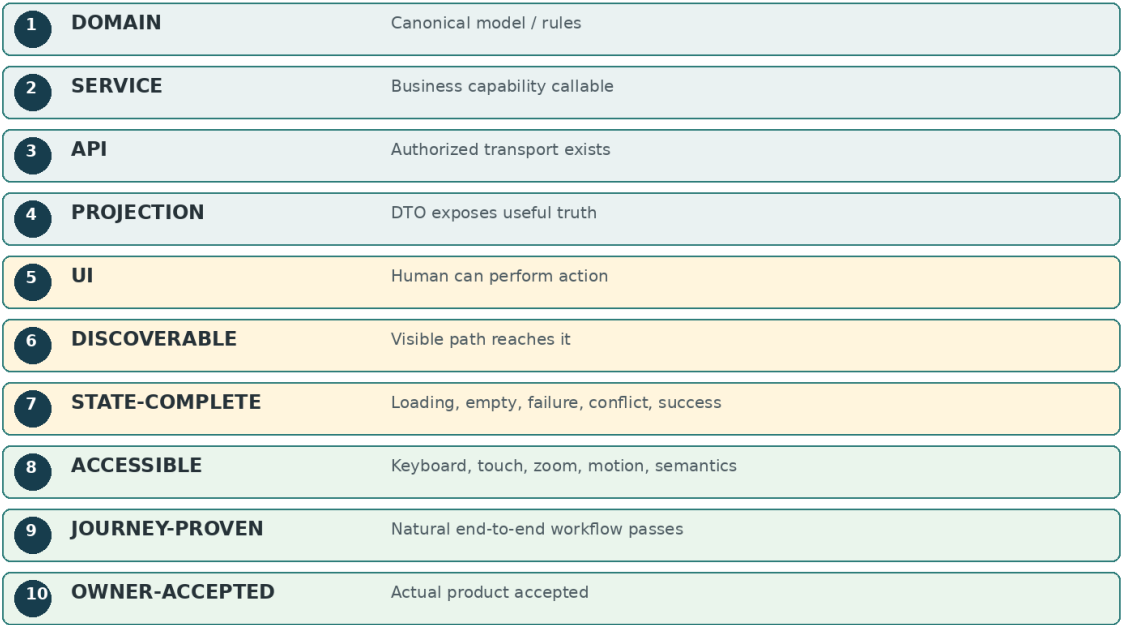
- Stable Deepwater capability ID, preferably mapped to a Feature Catalog feature/sub-capability ID.
- Canonical name and concise human meaning.
- Owning subsystem and contributing subsystems.
- User audience: visitor, Player, Captain, Creator, administrator, support operator, system-only, or external machine.
- Risk class and privacy class.
- Expected realization disposition.
- Current realization status and confidence.
- Evidence source and last audited commit.

8.2 Appropriate realization

Not every capability needs a full graphical interface. An internal retry scheduler may be INTERNAL_BY_DESIGN, while its health and failures may require an administrative projection. A cryptographic key should never be directly readable, while key version, rotation health, and provider status may be safely manageable in Admiralty. Deepwater records this distinction explicitly.

9. Capability Realization Ladder

Capability Realization Ladder



A backend capability is not a product feature merely because steps 1-4 exist.

Figure 3. The realization ladder prevents backend existence from being mistaken for product completion.

The ladder is not always linear. Some internal capabilities legitimately stop at service or API layers. User-facing capabilities normally must progress through discoverability, state completeness, accessibility, and journey proof. Deepwater records the expected terminal rung for each capability and compares it with the actual rung.

Rung	Meaning	Typical proof
DOMAIN	Canonical data/rule exists	Schema/domain tests
SERVICE	Business operation exists	Unit/service tests
API	Authorized transport exists	API/authorization tests
PROJECTION	Useful safe DTO exists	Projection/redaction tests
UI	Human surface exists	Component/browser evidence
DISCOVERABLE	Visible product path reaches it	Route/navigation journey
STATE-COMPLETE	Required states and feedback exist	Screen-state matrix
ACCESSIBLE	Keyboard/touch/zoom/motion/semantics work	Accessibility matrix
JOURNEY-PROVEN	Natural workflow succeeds end to end	Browser journey
OWNER-ACCEPTED	Running result accepted as product	Owner walkthrough record

10. Classification Taxonomy

Every audited capability receives one primary realization classification. These values are deliberately separate from implementation status such as IMPLEMENTED or RELEASE_VALIDATED.

Classification	Definition
FULLY_REALIZED	Capability reaches its required terminal rung with current evidence.
PARTIALLY_REALIZED	Some layers are complete, but one or more required layers or states are missing.
BACKEND_ONLY	Useful capability exists below the product surface with no appropriate human realization.
FRONTEND_ONLY	UI claims or simulates a capability that lacks canonical backend truth.
HIDDEN	Capability is usable only through non-obvious route, developer control, or undocumented path.
INTERNAL_BY_DESIGN	Capability is intentionally not exposed; internal realization is correct.
SECURITY_RESTRICTED	Capability is intentionally limited to privileged/support/admin surfaces.
MISSING	Governing capability exists conceptually but no meaningful implementation is present.
BROKEN	A previously intended realization is present but unusable, contradictory, or materially defective.
DEPRECATED	Capability remains only for bounded compatibility or historical support.

10.1 Secondary flags

- UNVERIFIED
- STALE_EVIDENCE
- OWNERSHIP_AMBIGUOUS
- LEGACY_DEPENDENT
- EXTERNAL_PROVIDER_PENDING
- VISUALLY_INCOMPLETE
- JOURNEY_UNPROVEN
- DOCUMENTATION_MISMATCH

11. Gap and Loss Taxonomy

Gap code family	Meaning
DW-CAP	Capability absent or materially narrower than governed intent
DW-SVC	Service/business operation missing or duplicated
DW-API	API/action exposure missing, unsafe, inconsistent, or non-idempotent
DW-PROJ	Projection/DTO omits useful safe truth or exposes forbidden truth
DW-UI	UI lacks appropriate control, view, hierarchy, or workflow
DW-NAV	Discoverability, navigation, parent/back/onward path gap
DW-STATE	Loading, empty, pending, conflict, success, recovery, offline, revoked, or unavailable state gap
DW-FEED	Weak or missing feedback after user action
DW-A11Y	Accessibility or alternative-interaction gap
DW-VIS	Visual maturity/design-system gap
DW-AUTH	Authorization/capability mismatch or privilege leak
DW-DOC	Feature Catalog/governing documentation mismatch
DW-OWN	Canonical ownership ambiguity or duplicate authority
DW-DEAD	Dead route/control/backend capability not intentionally retained

A finding may carry multiple gap families. For example, a device-session revocation endpoint with no UI and no confirmation would be DW-UI + DW-FEED; if reachable only by direct URL, also DW-NAV.

12. Canonical Capability Trace

A Deepwater trace is the core audit artifact. It follows one capability through every relevant implementation layer and records whether information, authority, or usability changes.

Capability: account.session.revoke

Owner: Wayfarer

Audience: account owner

DOMAIN -> AccountSession revocation state

SERVICE -> revokeSession(accountId, sessionId)

API -> POST /api/account/sessions/id/revoke

PROJECTION -> session device + status DTO

UI -> Sessions & Devices row action

DISCOVERY -> Account menu -> Personal Harbor -> Security -> Sessions

STATES -> idle / pending / revoked / current-session restriction / failure

A11Y -> keyboard button + live confirmation

JOURNEY -> sign in -> revoke secondary session -> verify disappearance

OWNER ACCEPTANCE -> accepted in running product

12.1 Trace rules

- Record exact source paths and stable identifiers when available.
- Do not infer a layer from naming alone; verify actual call paths.
- Record authorization at every mutation boundary.
- Record where DTOs sanitize or discard data.
- Record all product routes and entry points, not merely direct URLs.
- Record state transitions and user feedback.
- Record test/evidence IDs that prove each relevant layer.
- Record whether the capability's required terminal rung is appropriate for its audience.

13. Backend and Domain Audit Method

The backend survey begins with canonical domain and persistence truth, not UI assumptions. Auditors inventory models, state machines, service methods, event handlers, workers, provider adapters, projections, and privileged operations.

13.1 Required backend questions

- What meaningful operation or information exists here?
- Who is authorized to invoke or observe it?
- Is there one canonical implementation or multiple competing paths?
- Is the capability used by current production code or only tests/development tooling?
- Does it emit authoritative state, derived state, or audit state?
- Does it have failure/retry/cancellation semantics that the product should surface?
- Does it expose operational status that belongs in Admiralty rather than ordinary UI?
- Is the capability represented in Feature Catalog or governing documents?
- Are there accepted backend behaviors that have no named consumer?

13.2 Backend inventory sources

- Prisma schemas and migrations
- Domain/service directories
- Route handlers and server actions
- Outbox/worker/job handlers
- Authorization helpers
- Provider factories
- Audit events
- Feature flags and environment schemas
- Tests that imply hidden behavior

14. Service, API, and Action Audit Method

Deepwater distinguishes backend capability from transport exposure. A service may be correctly internal even when no public API exists. Conversely, an API route may duplicate domain logic and therefore represent an ownership defect rather than useful exposure.

Audit dimension	Questions
Coverage	Does every intended operation have an authorized call path?
Symmetry	Are create/read/update/delete or lifecycle operations exposed consistently where appropriate?
Mutation safety	Are idempotency, CSRF, concurrency, and revision conflicts represented?
Errors	Are stable machine codes and safe human messages available?
Pagination/filtering	Can large datasets be consumed without hidden hard limits?
Bulk operations	Does backend support safe bulk behavior that UI ignores, or vice versa?
Privileged operations	Should this be ordinary UI, Admiralty, support grant, or internal-only?
Legacy aliases	Do compatibility routes still own business logic?

15. Projection and DTO Audit Method

Projection loss is one of Deepwater's primary targets because it is easy for a rich backend to become a shallow product through over-simplified DTOs. Deepwater records both forbidden leakage and useful information loss.

- Compare canonical service result to public/owner/Captain/Creator/admin DTOs.
- Identify safe fields dropped without product reason.
- Identify raw internal fields passed through when an explicit projection should exist.
- Check historical snapshots versus current mutable profile data.
- Check enum/state collapse such as converting multiple failure reasons into a generic `error`.
- Check whether reason codes, provenance, timing, provider status, verification confidence, or lifecycle state could improve the human experience.
- Ensure secrets, private story content, raw object keys, tokens, security details, and forbidden participant data never cross the projection boundary.

Projection Rule
Deepwater never argues for 'more fields' in the abstract. It argues for the smallest safe projection that lets the intended user understand and use the capability fully.

16. Frontend and Interaction Audit Method

Frontend exposure is evaluated as behavior, not component existence. A screen with a label, raw form, or disabled button does not necessarily realize the backend capability.

16.1 Interaction completeness

- Can the intended user view the relevant state?
- Can the intended user perform every governed operation appropriate to their role?
- Are dangerous operations separated, confirmed, and reversible where possible?
- Does each action provide pending, success, and failure feedback?
- Are conflicts, stale data, revocation, offline, and retry states understandable?
- Does the UI preserve backend distinctions that matter to user decisions?
- Does the screen use established Voyagewright component and interaction patterns?
- Can the user leave, return, and recover without losing context?

16.2 False realization

A frontend-only mock, local state toggle, placeholder count, fake health indicator, or hardcoded success state is classified FRONTEND_ONLY or BROKEN when it claims a capability that the canonical system does not actually provide.

17. Discoverability and Navigation Audit Method

Deepwater consumes the authoritative route and navigation inventories established by Global Product Governance/Homeport. For every ordinary human-facing capability, the audit records at least one visible, logical, permission-aware entry path.

- Gateway or global navigation path.
- Workspace navigation path.
- Contextual card/list/detail path.
- Account/profile entry path.
- Dynamic detail source such as search, history, collection, or recommendation.
- Mobile-equivalent path.
- Return/back path and useful onward path.
- Permission-denied or sign-in path when the user lacks access.

A direct URL is evidence that the server knows a route. It is not evidence that the product realizes a feature.

18. Screen-State, Feedback, and Recovery Audit

Every realized screen must represent the states its backend can actually enter. Deepwater compares backend lifecycle/state enums and error codes against screen contracts to find collapsed or missing states.

State family	Examples
Data lifecycle	loading, populated, empty, no-result, removed, archived
Mutation	idle, pending, success, validation failure, authorization failure, conflict, retry
Session	anonymous, authenticated, expired, revoked, capability missing
Provider	configured, unconfigured, degraded, unavailable, stale
Network	online, weak, offline, reconnecting, resumed
Media	processing, ready, quarantined, failed, unavailable, fallback
Long-running operation	queued, claimed, running, paused, retrying, completed, terminal failure

When the backend has six meaningful states and the interface has 'works' and 'something went wrong,' Deepwater records a state-realization gap even if all API tests pass.

19. Accessibility, Responsive, and Motion Audit

A capability is not fully realized when it works only with a mouse, at one viewport, with motion enabled, or at 100% zoom. Deepwater does not replace specialist accessibility testing; it verifies that each capability has the evidence required by Global Product Governance and Sounding Line.

- Keyboard-only operation and visible focus.
- Touch target and mobile interaction parity.
- Effective 200% zoom and narrow viewport layout.
- Screen-reader names, descriptions, status announcements, and relationships.
- Reduced-motion behavior and static alternatives.
- Color-independent status communication.
- Media captions/transcripts/fallbacks when applicable.
- Accessible error and confirmation feedback.
- No feature available only through hover, drag, animation timing, or pointer precision.

20. Security, Privacy, and Restricted-Capability Audit

Deepwater must avoid turning audit enthusiasm into privilege escalation. Capabilities classified SECURITY_RESTRICTED or INTERNAL_BY_DESIGN are audited for correct administrative or operational realization, not broad exposure.

20.1 Restricted disposition examples

Capability	Expected disposition
Password credential hash	Internal only; never human-readable
Support account diagnostics	Admiralty with scoped user-approved support grant
KMS key material	Never readable; provider/key health may be manageable
Private object storage key	Internal; human surfaces use safe asset identity
Raw private Chronicle event payload	Owner-specific projection only where governed; never generic admin dump
Moderation evidence	Privileged moderation/support projection
Release deployment action	Admiralty/Breakwater privileged control with reauthentication and audit

20.2 Deepwater privacy proof

The ledger records privacy class and approved audience for every capability. Audit tooling must sanitize findings so the ledger itself cannot become a data-exfiltration artifact.

21. Documentation and Feature-Catalog Truth Audit

Deepwater reconciles product reality with the Feature Catalog and governing documents. A capability can be technically present while the catalog is stale, or cataloged as complete while the actual user surface is partial.

- Verify feature entry points against current route inventory.
- Verify feature maturity against current realization status.
- Verify project/phase attribution against actual canonical owner.
- Verify stated privacy and role boundaries.
- Verify current and historical version claims.
- Add uncataloged accepted capabilities when appropriate.
- Downgrade or annotate user-facing maturity when a capability is backend-only, hidden, or owner-walkthrough pending.
- Do not delete historical feature records merely because a capability becomes deprecated; record the transition.

Catalog Integrity Rule
A backend-only capability may be cataloged as infrastructure. A user-facing capability may not be represented as fully complete when its realization ladder stops before its governed terminal rung.

22. Capability Realization Ledger

The Capability Realization Ledger is the permanent machine-readable authority produced by Deepwater. It is not a spreadsheet somebody updates once and then forgets under `Development\_Docs/old-final-v2`.

22.1 Required ledger fields

Field group	Examples
Identity	capabilityId, featureCatalogId, name, ownerProject, ownerContract
Audience	roles, privacy class, privileged requirements
Expected realization	terminal rung, disposition, required surfaces
Layer trace	domain, service, API/action, projection, client, UI, route/navigation
State coverage	required states, represented states, missing states
Quality	discoverability, visual maturity, accessibility, responsive, recovery
Evidence	source SHA, test IDs, screenshots, journey IDs, owner acceptance
Classification	primary status, secondary flags, confidence
Gap	finding IDs, severity, impact, assigned owner, target phase
Lifecycle	first seen, last audited, closed at, regression status

22.2 Ledger storage

```
Development_Docs/Programs/Deepwater/  
  capability-realization-ledger.json  
  capability-realization-ledger.schema.json  
  findings/  
  evidence/  
  reports/  
  phase-records/
```

23. Finding, Assignment, and Ownership Workflow

Deepwater findings are actionable work items, not prose observations. Every open finding receives an owner, severity, confidence, affected capability, affected layer, expected remedy, verification requirement, and target integration vehicle.

1. Discover and reproduce the capability loss.
2. Create stable finding ID and sanitized evidence.
3. Identify canonical capability owner and contributing owners.
4. Determine whether the gap belongs to an already-active project phase.
5. Assign remediation to the owning project/phase or create a bounded Deepwater realization slice when the defect is truly cross-layer.
6. Define closure evidence before implementation begins.
7. Track the finding through current-main reconciliation.
8. Close only when the required realization rung and evidence are satisfied.
9. Update Feature Catalog and ledger if maturity changed.

23.1 Assignment destinations

Finding type	Typical destination
History presentation	Wakebook
Edition comparison	Tideglass
Captain operations	Helm
Creator Studio UX	Shipwright
Authoring validation contract	Drydock
Admin/support surface	Admiralty
Community exposure	Harborlight
Account/privacy capability	Wayfarer
Cross-layer route/projection glue with no better owner	Deepwater realization slice

24. Remediation Strategy and Deepwater Slices

Deepwater is primarily reconnaissance and governance, but some findings are integration defects that no single domain project naturally owns. These may be resolved through small Deepwater Realization Slices.

24.1 Deepwater slice eligibility

- The business capability already has a canonical owner and is not being redefined.
- The missing work is cross-layer glue, projection wiring, navigation integration, state mapping, or evidence.
- The fix can be independently mainline-safe.
- No active project phase is already rewriting the same surface.
- Schema changes are absent or minimal and explicitly coordinated.
- The slice has a clear Mainline Safety Contract and Sounding Line gate.

24.2 Deepwater must not use slices to

- Redesign another project's domain model
- Create replacement business services
- Introduce a second auth/session/progression authority
- Absorb major feature-roadmap work merely because the audit found it
- Bypass owner projects to move ledger rows to green

25. Cross-Project Coordination and Concurrency

Deepwater Phase 1 is intentionally Class A parallel work with very low schema pressure. It should run beside Drydock, Admiralty, Tideglass, Wakebook, Helm, and Shipwright without materially changing their source surfaces.

Later remediation is different. Deepwater must not modify a subsystem that is actively being rewritten by its owning project unless the projects deliberately coordinate a Class B interface. If the same canonical model or authority would be changed, serialize the work under Class C.

Concurrent condition	Deepwater rule
Owner project active on same UI	Audit freely; assign finding; wait to remediate unless coordinated
Owner project active on same schema	Do not edit schema; serialize
Gap is pure route/navigation metadata	May coordinate as Class B
Gap is cross-layer projection wiring	May become Deepwater slice if owner contracts are frozen
Finding depends on future owner capability	Record hard dependency; do not fabricate workaround
Main advances while audit is active	Refresh trace/evidence before final classification

Continuous Development Rule
Deepwater findings do not justify long-lived branches. Each implementation phase or realization slice follows the Continuous Development and Mainline Integration Standard: mainline-safe boundary, prompt convergence, fresh next-phase branch.

26. Sounding Line Verification and Evidence

Sounding Line remains the sole verification authority. Deepwater defines what realization must be proven; Sounding Line determines how the required evidence is selected, executed, and accepted.

26.1 Evidence families

- Static trace validation: source paths, owners, route inventory, ledger schema.
- Unit/service/API proof for backend and projection behavior.
- Component proof for screen-state mapping and interaction.
- Natural browser journeys beginning at real product entry points.
- Navigation reachability and mobile parity.
- Accessibility and effective-zoom evidence.
- Visual evidence for major user-facing realization closures.
- Negative authorization/privacy evidence.
- Owner walkthrough for major product-facing remediation.
- Post-merge source-bound proof on the integrated mainline SHA.

26.2 Evidence freshness

When main advances, Deepwater marks traces or evidence stale only where the changed source can affect them. Sounding Line decides which proof is invalidated. The ledger records source SHA and evidence state so stale screenshots cannot silently support current maturity.

27. Machine-Readable Development Control Plane

Deepwater must integrate with the repository's machine-readable governance rather than existing as an impressive PDF and a collection of human promises.

```
Development_Docs/Programs/Deepwater/
  capability-realization-ledger.json
  capability-realization-ledger.schema.json
  deepwater-findings.json
  deepwater-ownership-map.json
  deepwater-phase-status.json
  deepwater-audit-config.json
  evidence/
  reports/
```

Integration:

```
Development_Docs/Features/catalog/*.json
product/routes.json
product/screens.json
product/journeys.json
testing/contracts/product-*.json
Development_Docs/project-status.json
```

27.1 Policy validation

- Capability IDs unique and stable.
- Owners reference known projects/contracts.
- User-facing capabilities include a governed entry path.
- FULLY_REALIZED requires evidence for its expected terminal rung.
- Closed findings reference accepted evidence and source SHA.
- INTERNAL_BY_DESIGN and SECURITY_RESTRICTED include justification and expected admin/operational projection where applicable.
- Feature Catalog maturity cannot exceed Deepwater realization maturity without an explicit exception.

28. Metrics, Scoring, and Prioritization

Deepwater uses metrics to prioritize work, not to replace judgment. A single critical privacy gap matters more than fifty minor visual omissions.

28.1 Recommended metrics

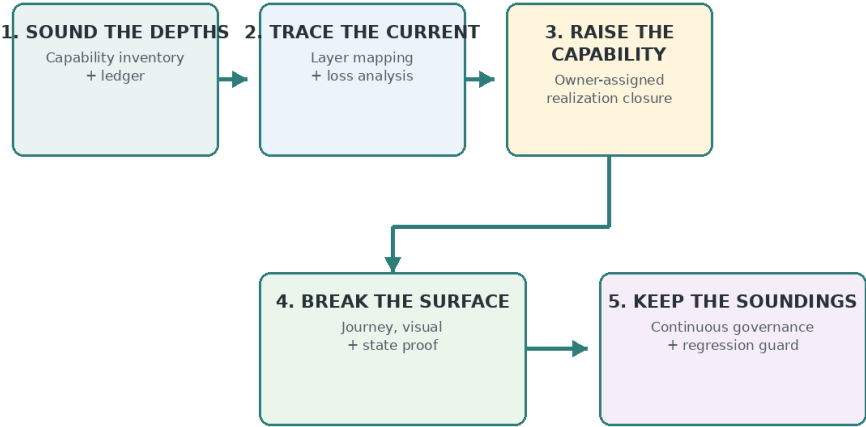
Metric	Purpose
Capability realization coverage	Percent of meaningful capabilities at expected terminal rung
Backend-only count	Useful capabilities stranded below product layer
Projection-loss count	Capabilities losing safe useful information at DTO boundary
Hidden capability count	Ordinary features requiring direct/developer paths
State completeness rate	Required screen states represented
Natural-journey coverage	Capabilities proven through visible-control journeys
Owner-accepted coverage	Major user-facing capabilities accepted in running product
Documentation mismatch count	Feature Catalog/governing claim divergence
Median finding age	How long capability gaps remain unresolved
Regression count	Previously closed capability gaps reopened

28.2 Priority score

A finding may receive a weighted score using user impact, severity, frequency, security/privacy risk, breadth, discoverability, implementation confidence, and whether it blocks other projects. Scores aid sequencing but never downgrade CRITICAL security/privacy/authorization findings.

29. Five-Phase Implementation Program

Project Deepwater Five-Phase Roadmap



Each phase is independently mainline-safe. A project's next phase starts only after the prior phase reaches accepted main.

Figure 4. Deepwater phases are independently mainline-safe and build from current integrated truth.

Phase	Formal scope	Mainline-safe result	Concurrency
Phase 1 - Sound the Depths	Complete Capability Inventory and Realization Ledger	Audit-only foundation; no broad source remediation	A
Phase 2 - Trace the Current	End-to-End Layer Mapping, Gap Root-Cause Analysis, and Owner Assignment	Adds complete traces and prioritized assignment packages	A/B
Phase 3 - Raise the Capability	Owner-Coordinated Realization Closure and Cross-Layer Slices	Closes assigned gaps without stealing canonical ownership	B/C per slice
Phase 4 - Break the Surface	Whole-Product Journey, Visual, State, Accessibility, and Recovery Proof	Proves realized capabilities through actual human-facing experience	B
Phase 5 - Keep the Soundings	Continuous Governance, Feature Catalog Integration, Regression Prevention, and Closure	Turns Deepwater into permanent drift detection and policy	A/B

29.1 Phase 1: Sound the Depths

Phase 1 inventories every meaningful capability from Feature Catalog, domain/service/API discovery, Homeport artifacts, and accepted project records. It creates the ledger, ownership map, trace skeletons, audit tooling, and initial classifications. It does not perform broad remediation.

- Revalidate current origin/main and Feature Catalog.
- Inventory domain services, APIs, DTOs, routes, screens, navigation, privileged surfaces, jobs, and provider seams.
- Assign stable capability IDs and canonical owners.
- Classify each capability using the primary taxonomy.
- Create initial finding records for obvious backend-only, hidden, broken, or documentation-mismatch cases.
- Generate coverage metrics and a prioritized Phase 2 trace queue.
- Update no product behavior unless a tiny audit-tooling dependency is required.

Phase 1 Mainline Safety Contract

After Phase 1, Voyagewright behaves exactly as before, but the repository gains an authoritative capability-realization ledger and audit tooling. Schema changes should be zero unless a separately approved control-plane need is discovered. If Deepwater ends forever here, main remains coherent.

29.2 Phase 2: Trace the Current

Phase 2 fully traces priority capabilities through all relevant layers, determines the exact point of loss, and assigns each gap to its canonical owner. It creates remediation packets with closure evidence rather than implementing everything itself.

Phase 2 Mainline Safety Contract

Main gains precise trace and assignment data. Product behavior remains stable except for safe audit instrumentation. Findings can be consumed by other projects independently.

29.3 Phase 3: Raise the Capability

Phase 3 coordinates realization closure. Most work occurs in the owning projects and their phases. Deepwater implements only eligible cross-layer realization slices. Each slice is integrated promptly rather than stacking a giant Deepwater remediation branch.

Phase 3 Mainline Safety Contract

Each remediation slice is independently complete and mainline-safe. No slice assumes another future slice will make current behavior coherent.

29.4 Phase 4: Break the Surface

Phase 4 performs whole-product proof for capabilities classified user-facing: natural navigation, state completeness, responsive behavior, accessibility, error recovery, meaningful demo data, and visual/product quality. Major changes stop at READY FOR OWNER WALKTHROUGH until the owner accepts them.

Phase 4 Mainline Safety Contract

Only coherent, journey-proven user-facing improvements enter main. Owner acceptance status is preserved separately from automated acceptance.

29.5 Phase 5: Keep the Soundings

Phase 5 institutionalizes Deepwater. New Feature Catalog entries, routes, screens, APIs, and project completion records are checked against capability-realization policy. Regression detection prevents a future backend expansion from silently outrunning the product again.

Phase 5 Mainline Safety Contract

Deepwater becomes a permanent governance and drift-detection system. Removal of the program would not be required for product function, but would remove a release-quality safeguard.

30. Final Program Acceptance and Continuous Governance

Project Deepwater is not complete merely because the initial audit has a large number of rows. Program closure requires a trusted ledger, explicit ownership for every meaningful capability, closure or accepted disposition of all critical/high realization gaps, and automated regression checks that keep the ledger relevant.

30.1 Final acceptance criteria

- Every current Feature Catalog capability is represented or explicitly mapped to a parent capability in the Deepwater ledger.
- All significant uncataloged backend capabilities discovered by the audit have an intentional disposition.
- No user-facing capability is classified FULLY_REALIZED without appropriate discoverability, state, accessibility, journey, and evidence requirements.
- All CRITICAL and HIGH findings are closed, accepted as explicit debt with owner/expiry, or blocked by a documented external dependency.
- No open finding has ambiguous canonical ownership.
- All major user-facing remediation has current owner-walkthrough status where required.
- Feature Catalog maturity and Deepwater realization status are reconciled.
- Sounding Line validates ledger schema, evidence references, route/screen relationships, and regression rules.
- New project prompts are required to declare capability-realization impact.
- Deepwater Phase 1 audit can be rerun deterministically against a new mainline and produce explainable deltas.

Final Governing Rule
Useful capability may not disappear between backend and human without an intentional, documented reason. Deepwater makes that loss visible, owned, actionable, and testable.

Appendix A. Capability Realization Ledger Schema

```
{
  "capabilityId": "DW-CAP-ACCOUNT-SESSION-REVOKE",
  "featureCatalogId": "FT-B00X",
  "name": "Revoke an account session",
  "ownerProject": "Wayfarer",
  "audience": ["ACCOUNT_OWNER"],
  "privacyClass": "PRIVATE_OWNER",
  "expectedTerminalRung": "JOURNEY_PROVEN",
  "expectedDisposition": "USER_FACING",
  "classification": "FULLY_REALIZED",
  "secondaryFlags": [],
  "trace": {
    "domain": ["AccountSession"],
    "service": ["revokeSession"],
    "transport": ["POST /api/account/sessions/:id/revoke"],
    "projection": ["AccountSessionDeviceDTO"],
    "ui": ["Personal Harbor / Sessions & Devices"],
    "navigation": ["account menu -> security -> sessions"]
  },
  "states": ["idle","pending","success","failure","current-session-restricted"],
  "evidence": {
    "sourceSha": "...",
    "tests": ["..."],
    "journeys": ["..."],
    "screenshots": ["..."]
  },
  "findings": [],
  "lastAuditedAt": "..."
}
```

Validation rule	Requirement
Stable identity	Capability ID never changes merely because files move.
Known owner	Owner must reference canonical project/contract.
Terminal rung	Required for every capability.
User-facing classification	Requires navigation and screen evidence.
Fully realized	Cannot coexist with open blocking finding.
Restricted/internal	Must include rationale and safe operational projection if one is required.
Evidence	Must be source-bound and current.

Appendix B. Capability Trace Worksheet

Layer	Required audit entry
Capability meaning	What can the platform do and why does it matter?
Owner	Who owns the business truth?
Domain	Models/state/events defining truth
Service	Canonical callable behavior
Transport	API/server action/worker/provider interface
Authorization	Who may invoke or observe?
Projection	Safe DTOs for each audience
Client	Data-fetch/state layer
UI	Screens/components/actions
Discoverability	Visible path from gateway/workspace/context
States	Loading, empty, pending, failure, conflict, offline, success, removed, etc.
Accessibility	Keyboard, touch, zoom, semantics, reduced motion
Journey	Natural end-to-end proof
Evidence	Tests, screenshots, receipts, owner walkthrough
Loss point	First layer where intended capability is reduced or absent

Appendix C. Finding and Assignment Record

Finding ID: DW-031

Capability: ...

Gap families: DW-PROJ, DW-UI

Severity: HIGH

Confidence: HIGH

Observed on source: <SHA>

Evidence: <paths / journey / screenshot IDs>

Expected behavior:

...

Current behavior:

...

First loss point:

...

Canonical owner:

...

Contributing owners:

...

Assigned project/phase:

...

Mainline dependency:

...

Closure evidence:

...

Status:

OPEN | ASSIGNED | IN_PROGRESS | MAINLINE_CANDIDATE | CLOSED | DEBT_ACCEPTED

- Never place raw secrets, private Chronicle prose, invitation credentials, or unnecessary personal data in a finding.
- Do not close a finding merely because source code changed; close it when required evidence on accepted mainline exists.
- Debt acceptance requires owner, reason, expiry/trigger, and compensating behavior.

Appendix D. Phase 1 Audit Checklist

- Fetch current origin/main and record exact SHA.
- Read current Feature Catalog and validate generated source.
- Load Global Product Governance route/screen/journey inventories.
- Enumerate canonical subsystem owners and active phases.
- Inventory Prisma models and domain services.
- Inventory route handlers, server actions, and privileged/internal endpoints.
- Inventory DTOs/projections and sanitizers.
- Inventory human-facing routes and components.
- Inventory global/workspace/context navigation.
- Inventory background jobs, provider seams, operations, and health data.
- Map every catalog capability to an initial trace.
- Discover uncataloged meaningful capabilities.
- Apply primary classification and secondary flags.
- Create critical/high obvious findings without attempting broad remediation.
- Generate summary metrics and owner breakdown.
- Validate ledger schema and referential integrity.
- Run Sounding Line focused audit gate.
- Reconcile against latest main before acceptance.
- Merge Phase 1 promptly when accepted; do not begin Deepwater Phase 2 on the unmerged Phase 1 branch.

Appendix E. Example Capability Traces

E.1 Artifact provenance

Expected owner: Wayfarer for personal artifact history; One Voyage for authoritative grant truth. Deepwater should trace grant receipt -> Wayfarer projection -> Artifact Cabinet API -> artifact detail UI -> historical provenance display -> navigation from Personal Harbor -> owner-only notes/privacy -> natural Voyage-history journey.

E.2 Community remix lineage

Expected owner: Harborlight. Trace immutable release lineage -> service -> public-safe projection -> listing/release UI -> compare/source links -> discoverability from Community Harbor -> unavailable/removed-parent states -> attribution accessibility.

E.3 Captain event history

Expected owner: Helm for operational presentation, One Voyage for event truth. Trace canonical event sequence -> Captain projection -> event stream UI -> filters/context -> warning states -> permissions -> participating-Captain safe separation.

E.4 Release/provider health

Expected owner: Breakwater/Sealed Hold for provider truth and Admiralty for privileged control surface. Deepwater should not expose raw credentials; it should verify health, configuration state, failures, last successful operation, and actionable remediation are appropriately projected.

Appendix F. Mainline Safety Contracts by Phase

Phase	Schema impact	Active behavior after merge	If later phase never happens
1. Sound the Depths	None expected	Ledger/audit tooling available; product unchanged	Repository has a useful permanent capability inventory
2. Trace the Current	None expected	Detailed traces and owner assignments	Owners can act on findings independently
3. Raise the Capability	Per-owner/slice; coordinated	Each closed slice is fully usable	All merged slices remain coherent improvements
4. Break the Surface	Minimal; only owner-approved	Journey/state/visual closures active	Product remains improved and evidence-bound
5. Keep the Soundings	Governance metadata only	Continuous drift/regression checks active	Product still functions; governance safety remains complete at last accepted state

Appendix G. Final Acceptance Checklist

- Current-main source identity recorded.
- Capability ledger validates against its schema.
- Every capability has a canonical owner and expected terminal rung.
- Every Feature Catalog entry is reconciled.
- Uncataloged meaningful capabilities have intentional disposition.
- All critical/high findings are closed, accepted debt, or externally blocked with exact evidence.
- No ordinary user-facing capability remains hidden without explicit governing exception.
- No capability is falsely represented as complete by documentation.
- No audit remediation created duplicate business authority.
- All active Deepwater source changes are in accepted main; no stacked Deepwater phase branch remains.
- Sounding Line mainline decision passes for the integrated closure.
- Major human-facing remediation has current owner-walkthrough status.
- Continuous regression policy is active.
- Program completion receipt points to the exact ledger digest and source SHA.

Closure Statement
Project Deepwater is complete when Voyagewright can account for the fate of its capabilities from domain truth to human experience, and when future development is structurally prevented from letting those capabilities vanish beneath the surface again.